

EXPERIMENT – 4

Name: K ANUSRI

Roll No: 2520030432

Section: 8

Title

Multiple Child Processes using wait() and waitpid() and Observation of Zombie Processes

Aim

To create multiple child processes using fork(), use wait()/waitpid() for synchronization, and observe and eliminate zombie processes.

Theory

- fork() creates a new child process.
- Multiple child processes can be created from a parent process.
- wait() allows the parent to wait for a child process to terminate.
- waitpid() allows the parent to wait for a specific child process.
- A **zombie process** is a terminated child process that has not yet been collected by its parent.
- Calling wait() or waitpid() collects the terminated child and eliminates the zombie state.

Algorithm

1. Start the program.
2. Create the first child process using fork().
3. Display the PID and PPID of Child 1.
4. Create the second child process using fork().
5. Display the PID and PPID of Child 2.
6. Display the parent PID and both child PIDs.
7. Allow the child processes to terminate.
8. Observe the zombie process using the ps command.
9. Use waitpid() to collect Child 1 and Child 2.
10. Observe that the zombie processes are eliminated.
11. Terminate the parent process.

Program Code

```
k_anusri@DESKTOP-TM20BA... × + ▾
k_anusri@DESKTOP-TM20BAQ:~$
k_anusri@DESKTOP-TM20BAQ:~$ cat > exp4_multiple_children.c <<'EOF'
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t child1, child2;

    child1 = fork();

    if (child1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (child1 == 0)
    {
        printf("Child 1\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
        exit(0);
    }

    child2 = fork();

    if (child2 < 0)
    {
        perror("fork");
        return 1;
    }

    if (child2 == 0)
    {
        printf("Child 2\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
        exit(0);
    }

    printf("Parent Process\n");
    printf("Parent PID : %d\n", getpid());
    printf("Child 1 PID: %d\n", child1);
    printf("Child 2 PID: %d\n", child2);

    sleep(10);

    waitpid(child1, NULL, 0);
    waitpid(child2, NULL, 0);

    printf("Child processes collected.\n");
    printf("Parent process completed.\n");

    return 0;
}
EOF
```

Execution and Screenshots

Screenshot 1 — Compilation and Execution

```
k_anusri@DESKTOP-TM20BAQ:~$ gcc exp4_multiple_children.c -o exp4_multiple_children
k_anusri@DESKTOP-TM20BAQ:~$ ./exp4_multiple_children &
[1] 1012
Parent Process
Child 1
Parent PID : 1012
Child 1 PID: 1014
Child 2 PID: 1015
PID : 1014
PPID : 1012
Child 2
PID : 1015
PPID : 1012
```

Screenshot 2 — Zombie Process Observation

```
k_anusri@DESKTOP-TM20BAQ:~$ ps -el | grep Z
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
1 Z   1000      1014     1012  0  80   0 -    0 -    pts/0    00:00:00 exp4_multiple_c
1 Z   1000      1015     1012  0  80   0 -    0 -    pts/0    00:00:00 exp4_multiple_c
```

Screenshot 3 — Process Tree / Process State

```
k_anusri@DESKTOP-TM20BAQ:~$ Child processes collected.
Parent process completed.
ps -o pid,ppid,state,cmd --forest
  PID     PPID S  CMD
    959     953 S  -bash
   1039     959 R  \_ ps -o pid,ppid,state,cmd --forest
[1]+  Done                  ./exp4_multiple_children
```

Screenshot 4 — Zombie Elimination

```
k_anusri@DESKTOP-TM20BAQ:~$ wait
k_anusri@DESKTOP-TM20BAQ:~$ ps -el | grep Z
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
```

Result

Multiple child processes were successfully created using `fork()`. The child processes were observed in the zombie state, and `waitpid()` was used to collect the terminated child processes and eliminate the zombie state.